```
%% SEG800 Assignment - Comb Filter Demo
% This script demonstrates two common comb filters:
% 1. A feedforward comb filter
% 2. A feedback comb filter
%
% The script is written as a single file so it is easy to open in MATLAB,
% run section by section, convert to a live script if needed, and export to
% PDF for submission.
%
% How to use:
% 1. Open this file in MATLAB.
% 2. Run the script.
% 3. Review the figures and the printed notes in the Command Window.
% 4. Save as .mlx and export to PDF if your group wants a live script.

clear; close all; clc;

%% Setup
% Keep the parameter names descriptive so the filter behavior is easy to
% follow.

sampleRate = 8000;          % Samples per second
durationSeconds = 0.05;     % Short duration is enough for this demo
delaySamples = 40;          % Delay used by both comb filters
feedforwardGain = 1.0;      % Gain for y[n] = x[n] + a*x[n-D]
feedbackGain = 0.7;         % Gain for y[n] = x[n] + a*y[n-D]

timeVector = 0:1/sampleRate:durationSeconds - 1/sampleRate;
frequencySpacing = sampleRate / delaySamples;
firstFeedforwardNotch = sampleRate / (2 * delaySamples);

fprintf('SEG800 Comb Filter Demo\n');
```

```
SEG800 Comb Filter Demo
```

```
fprintf('Sample rate: %d Hz\n', sampleRate);
```

```
Sample rate: 8000 Hz
```

```
fprintf('Delay: %d samples\n', delaySamples);
```

```
Delay: 40 samples
```

```
fprintf('Comb spacing: %.2f Hz\n', frequencySpacing);
```

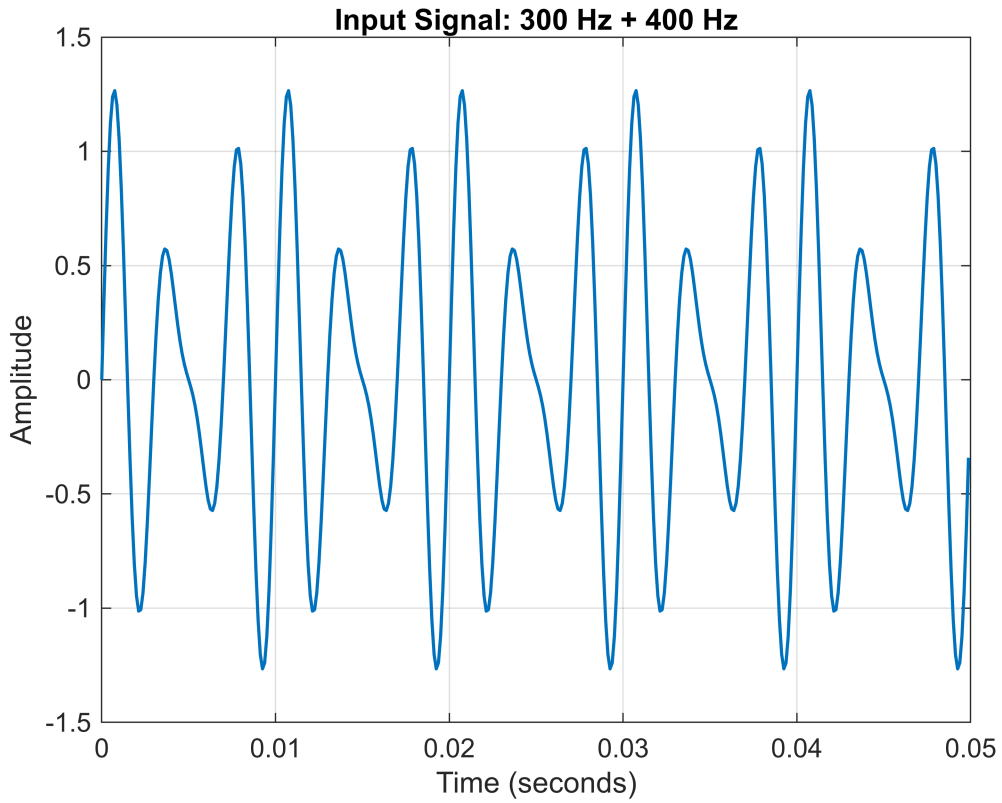
```
Comb spacing: 200.00 Hz
```

```
fprintf('First feedforward notch for a = 1: %.2f Hz\n\n', firstFeedforwardNotch);
```

```
First feedforward notch for a = 1: 100.00 Hz
```

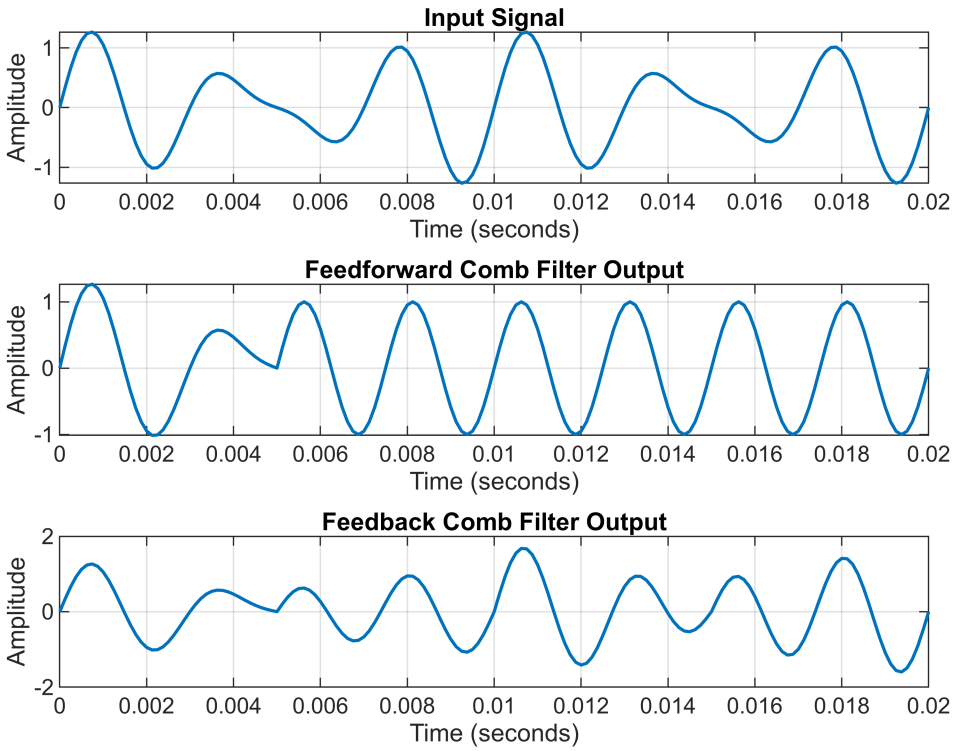
```

%% Create a simple test signal
```

```
% The test signal uses two tones:
% - 300 Hz is close to a feedforward notch for this delay.
% - 400 Hz is between notches and should remain stronger.

notchToneFrequency = 300;
passToneFrequency = 400;

inputSignal = 0.8 * sin(2 * pi * notchToneFrequency * timeVector) ...
    + 0.5 * sin(2 * pi * passToneFrequency * timeVector);

figure('Name', 'Input Signal');
plot(timeVector, inputSignal, 'LineWidth', 1.2);
grid on;
xlabel('Time (seconds)');
ylabel('Amplitude');
title('Input Signal: 300 Hz + 400 Hz');
```



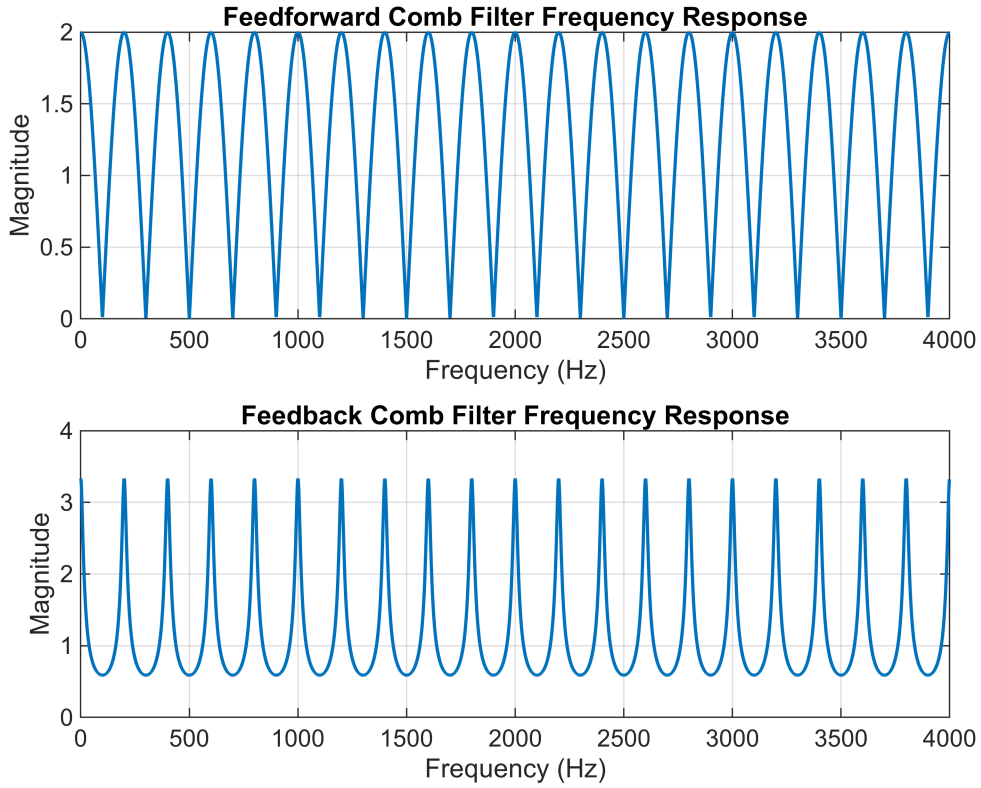

```
%% Feedforward comb filter
% Difference equation:
%   y[n] = x[n] + a * x[n-D]
%
% This version creates regularly spaced peaks and notches because the input
% is added to a delayed copy of itself.

[feedforwardNumerator, feedforwardDenominator] = ...
    createFeedforwardCombCoefficients(delaySamples, feedforwardGain);
```

```
feedforwardOutput = filter(feedforwardNumerator, feedforwardDenominator,
inputSignal);

%% Feedback comb filter
% Difference equation:
%   y[n] = x[n] + a * y[n-D]
%
% Rearranged for MATLAB filter form:
%   y[n] - a * y[n-D] = x[n]
%
% This version produces a stronger resonant effect because past output
% samples are fed back into the system.

[feedbackNumerator, feedbackDenominator] = ...
    createFeedbackCombCoefficients(delaySamples, feedbackGain);

feedbackOutput = filter(feedbackNumerator, feedbackDenominator, inputSignal);

%% Plot time-domain comparison
% Showing the first 20 ms keeps the plots readable.

timeWindowMask = timeVector <= 0.02;

figure('Name', 'Time-Domain Comparison');
subplot(3, 1, 1);
plot(timeVector(timeWindowMask), inputSignal(timeWindowMask), 'LineWidth', 1.2);
grid on;
title('Input Signal');
xlabel('Time (seconds)');
ylabel('Amplitude');

subplot(3, 1, 2);
plot(timeVector(timeWindowMask), feedforwardOutput(timeWindowMask), 'LineWidth',
1.2);
grid on;
title('Feedforward Comb Filter Output');
xlabel('Time (seconds)');
ylabel('Amplitude');

subplot(3, 1, 3);
plot(timeVector(timeWindowMask), feedbackOutput(timeWindowMask), 'LineWidth', 1.2);
grid on;
title('Feedback Comb Filter Output');
xlabel('Time (seconds)');
ylabel('Amplitude');
```

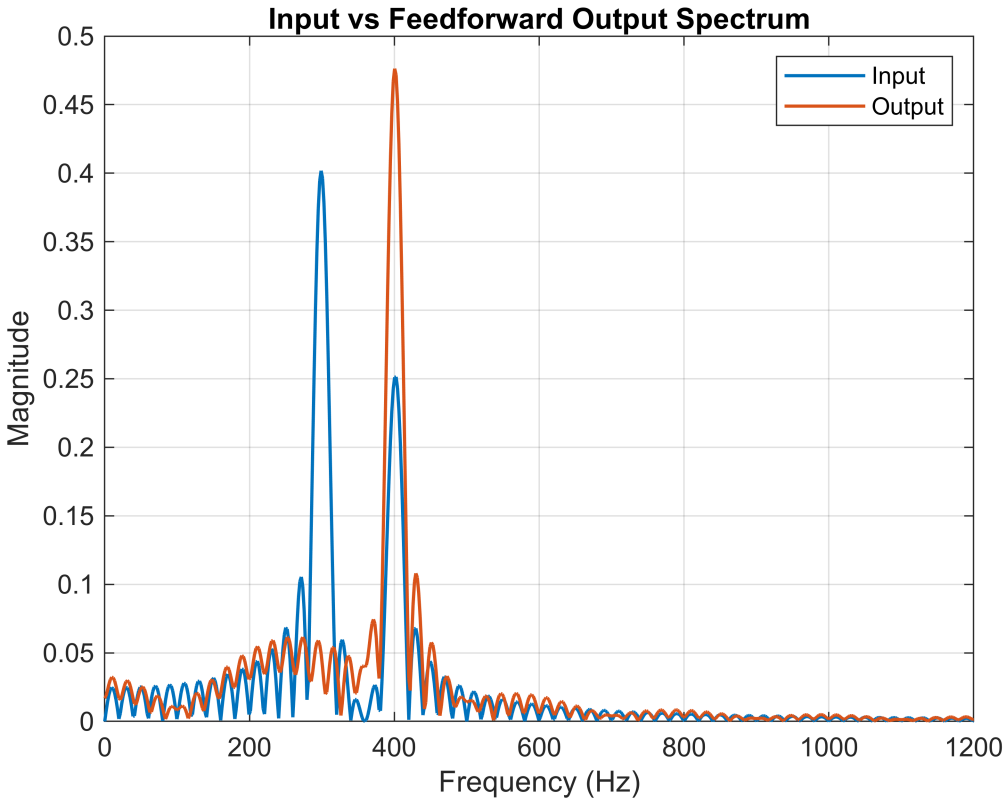



```
%% Plot frequency response of both filters
% We plot the magnitude response directly so the comb pattern is easy to
% compare in one figure.

responsePoints = 4096;

figure('Name', 'Comb Filter Frequency Responses');
subplot(2, 1, 1);
[feedforwardResponse, frequencyAxis] = freqz( ...
    feedforwardNumerator, feedforwardDenominator, responsePoints, sampleRate);
plot(frequencyAxis, abs(feedforwardResponse), 'LineWidth', 1.2);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([0 sampleRate / 2]);
title('Feedforward Comb Filter Frequency Response');

subplot(2, 1, 2);
[feedbackResponse, frequencyAxis] = freqz( ...
    feedbackNumerator, feedbackDenominator, responsePoints, sampleRate);
plot(frequencyAxis, abs(feedbackResponse), 'LineWidth', 1.2);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([0 sampleRate / 2]);
```

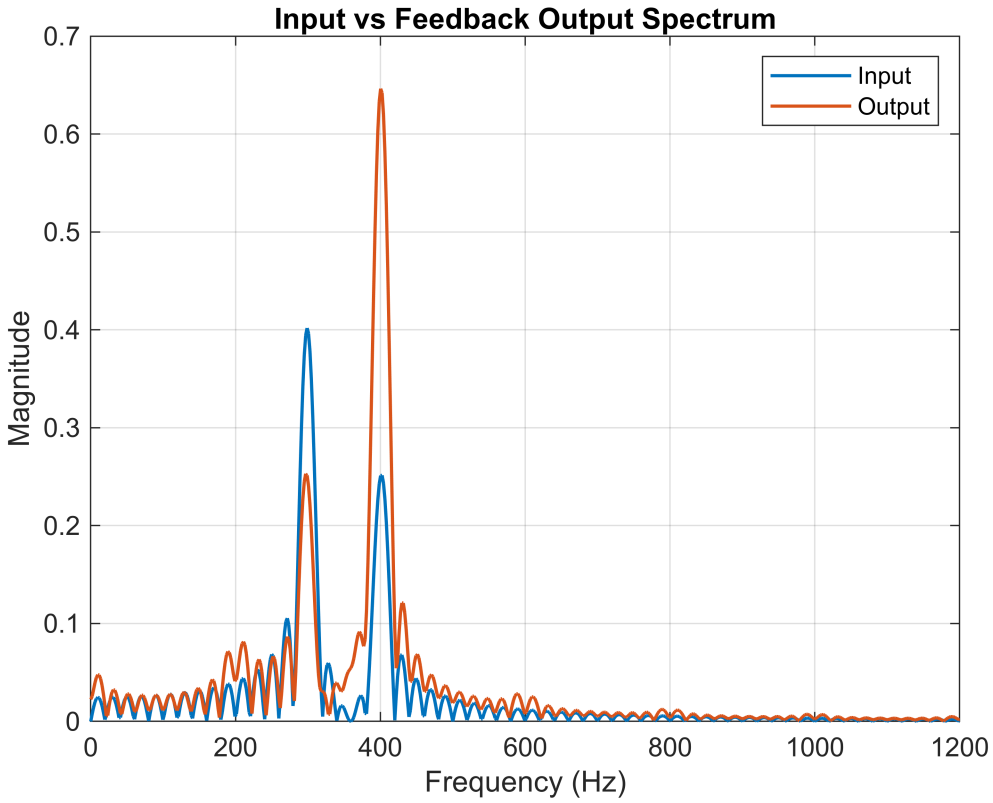
```
title('Feedback Comb Filter Frequency Response');
```




```
%% Plot output spectra
% These plots help confirm that the two filters shape the same input signal
% in different ways.

plotSpectrumComparison(inputSignal, feedforwardOutput, sampleRate, ...
    'Input vs Feedforward Output Spectrum');
```




```
plotSpectrumComparison(inputSignal, feedbackOutput, sampleRate, ...
    'Input vs Feedback Output Spectrum');
```




```
%% Print short interpretation notes
fprintf('Interpretation notes:\n');
```

```
Interpretation notes:
```

```
fprintf('- Comb spacing is controlled by sampleRate / delaySamples.\n');
```

```
- Comb spacing is controlled by sampleRate / delaySamples.
```

```
fprintf('- The feedforward filter creates repeated notches and peaks.\n');
```

```
- The feedforward filter creates repeated notches and peaks.
```

```
fprintf('- The feedback filter creates sharper resonances.\n');
```

```
- The feedback filter creates sharper resonances.
```

```
fprintf('- Changing delaySamples changes the spacing of the comb teeth.\n');
```

```
- Changing delaySamples changes the spacing of the comb teeth.
```

```
fprintf('- Changing the gain changes how strong the comb effect becomes.\n');
```

```
- Changing the gain changes how strong the comb effect becomes.
```

```
%% Local functions
```

```
function [numerator, denominator] = createFeedforwardCombCoefficients(delaySamples,
gainValue)
    validateattributes(delaySamples, {'numeric'}, {'scalar', 'integer',
'positive'});
    validateattributes(gainValue, {'numeric'}, {'scalar', 'real'});

    numerator = zeros(1, delaySamples + 1);
    numerator(1) = 1;
    numerator(end) = gainValue;
    denominator = 1;
end

function [numerator, denominator] = createFeedbackCombCoefficients(delaySamples,
gainValue)
    validateattributes(delaySamples, {'numeric'}, {'scalar', 'integer',
'positive'});
    validateattributes(gainValue, {'numeric'}, {'scalar', 'real', '<', 1, '>', -1});

    numerator = 1;
    denominator = zeros(1, delaySamples + 1);
    denominator(1) = 1;
    denominator(end) = -gainValue;
end

function plotSpectrumComparison(inputSignal, outputSignal, sampleRate, figureTitle)
    spectrumPoints = 4096;

    [inputMagnitude, frequencyAxis] = computeSingleSidedSpectrum(inputSignal,
sampleRate, spectrumPoints);
    [outputMagnitude, ~] = computeSingleSidedSpectrum(outputSignal, sampleRate,
spectrumPoints);

    figure('Name', figureTitle);
    plot(frequencyAxis, inputMagnitude, 'LineWidth', 1.2);
    hold on;
    plot(frequencyAxis, outputMagnitude, 'LineWidth', 1.2);
    hold off;
    grid on;
    xlim([0 1200]);
    xlabel('Frequency (Hz)');
    ylabel('Magnitude');
    title(figureTitle);
    legend('Input', 'Output', 'Location', 'northeast');
end

function [magnitudeSpectrum, frequencyAxis] =
computeSingleSidedSpectrum(signalData, sampleRate, spectrumPoints)
    fftValues = fft(signalData, spectrumPoints);
    halfPoint = floor(spectrumPoints / 2) + 1;
```

```
    magnitudeSpectrum = abs(fftValues(1:halfPoint));
    magnitudeSpectrum = magnitudeSpectrum / numel(signalData);
    frequencyAxis = linspace(0, sampleRate / 2, halfPoint);
end
```